

TEKNOFEST

HAVACILIK, UZAY VE TEKNOLOJİ FESTİVALİ

SAĞLIKTA YAPAY ZEKÂ YARIŞMALARI

(Bilgisayarlı Görüyle Abdomen (Karın) Bölgesi için

Hastalık Tespiti Kategorisi)

PROJE DETAY RAPORU

TAKIM ADI: BUROBOT

TAKIM ID: 359713

İçindekiler

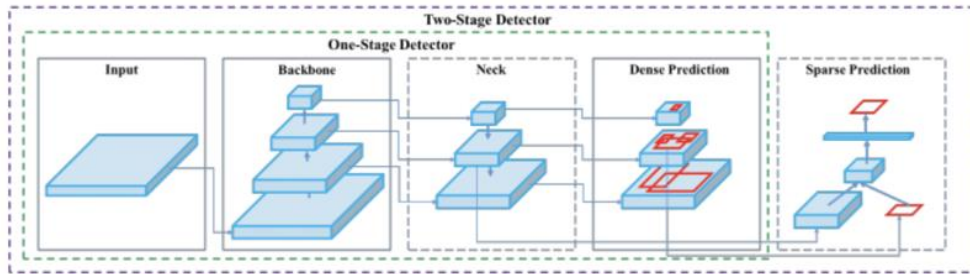
1. Proje Mevcut Durum Değerlendirmesi	3
2. Özgünlük.....	4
3. Sonuçlar ve İnceleme	5
4. Deney ve Eğitim Aşamalarında Kullanılan Veri Setleri	9
4.1. Verisetinin Eğitime Hazırlanması	9
5. Referanslar	12

1. Proje Mevcut Durum Değerlendirmesi

BuTespit ismini verdiğimiz projemizde, çalışma sürecimize, yarışma için hangi yapay zeka modelini geliştireceğiz ve verilen verisetini nasıl eğiteceğiz? temel sorusundan hareketle çalışmaya başladık. Bu soru bizi 2 farklı yönteme sürükledi. Bunlardan birisi Segmentasyon (Görüntü Bölütleme) ve diğeri ise Nesne Algılama (Object Detection) oldu. Her iki konu hakkında araştırma yaptık ve projemize Nesne Algılama ile başlamaya karar verdik. Proje detay raporunun yetişmesi gereken zamanın kısıtlı olması ve bizim öğrenmemiz gereken birçok kavramın bulunmasından dolayı segmentasyon konusunun öğrenilmesi için gereken zamanı bulamayabildik. Bu yüzden Nesne Algılama Yönteminin daha öncelikli olduğunu düşünerek bu konu hakkında çalışmaya karar verdik. Eğer yeterli zamanı bulursak Segmentasyon çalışmalarını da projemize eklemeyi karar verdik.

Nesne Algılama algoritmalarını incelediğimizde bunların Baseline ve Backbone olmak üzere iki sınıfa ayrıldığını gördük. YOLO modeli nesne algılama problemini doğrudan görüntü piksellerinden sınırlayıcı kutu koordinatlarına ve olasılık sınıflarına kadar tek bir regresyon sorunu olarak değerlendirmektedir. YOLO mimarisinde ağıın ilk evrişimsel katmanları görüntüden öznitelikleri çıkarırken tamamen bağlı katmanlar çıktı olasılıklarını ve koordinatlarını tahmin eder. YOLO, tam görüntüler üzerinde eğitim alır ve algılama performansını doğrudan optimize eder. Bu birleşik model, iki aşamalı modellerle kıyaslandığında; hız, daha az arka plan hatası yapma ve daha genellenebilir çıkarımda bulunma gibi özellikleri ile ön plana çıkmaktadır Ayrıca nesneleri diğer tanıma sistemlerine göre daha hızlı ve daha kesin olarak tanımlar [1,2].

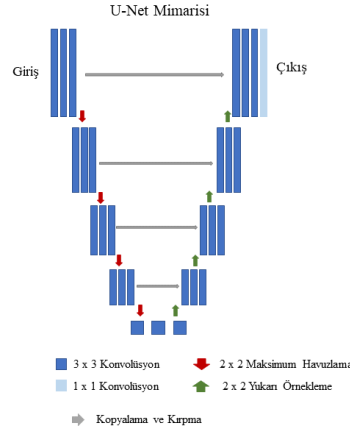
YOLOv4 modeli; kafa olarak kendisinden önce geliştirilmiş YOLOv3 modeli, omurga olarak CSPDarknet53 mimarisi ve boyun olarak SPP ve PAN kullanılarak geliştirilmiştir. Geliştirilen modelin ImageNet ve MS COCO veri seti ile eğitimi sırasında etkinliğini artırmak için farklı veri artırma, sınıf etiketi yumuşatma, aktivasyon fonksiyonlarını değiştirme gibi teknikler uygulanmıştır. Geliştirilen modelin diğer yeni teknoloji gerçek zamanlı nesne algılama modellerine göre daha hızlı ve başarılı olduğu tespit edilmiştir [2].



Şekil 1. Yolo v4 Mimarisi

Bu nedenlerden dolayı bu proje çalışmamızda çözüm için Baseline algoritmaları içinden derin öğrenme algoritması olan Yolo v4 ve backbone algoritmalarından DarkNet in beraber çalıştığı bir modeli benimsedik.

Segmentasyon için biyomedikal görüntü bölütleme alanında son derece iyi performans gösteren bir derin öğrenme modeli olan UNet mimarisi kullanılması öngörülmektedir. Bu mimari ön tasarım raporunda da belirtilmiştir. Mimari temelde iki kol içerir, ve bu kollar U harfine benzer bu sebeple UNet ismini almıştır. İlk kol görüntüdeki bağlamı yakalamak için oluşturulan kodlayıcı (enkoder) koludur. Konvolüsyon ve maksimum havuzlama katmanlarından oluşur. İkinci kol ise transpoze evrişimleri kullanarak tespit yapmak için oluşturulan kod çözücü (decoder) koludur [3]. UNet modelinin mimarisi Şekil 1’de gösterilmektedir UNet mimarisinde transfer öğrenmeyi de kullanabilmek için deneylerde en iyi sınıflama sonucunu veren evrişimli ağıın ağırlık ve katmanları U-net’ in encoder(kodlayıcı) bölümünde yer alacak, sonra upconvolution ve convolution işlemleriyle yarışma sürecine kadar bölütleme görüntüsü elde edilmesi planlanmaktadır.



Şekil 2. U-Net Modeli

Eğitim çalışmalarına Google Colab ismiyle bilinen colab.research.google.com adresi üzerinde modelimizi geliştirmeye başladık. Google Colab (Google Colaboratory), yapay zeka ve derin öğrenme projeleri üzerinde çalışanlar için etkileşimli, tamamen bulut tabanlı, kullanımı kolay ve ortak çalışmaya dayalı bir programlama ortamıdır [4]. Çevrimiçi bir ortam olarak Google Colab'ı seçme nedenlerimiz; Ücretsiz olması, kurulum gerektirmemesi, en kapsamlı yapay sinir ağları, algoritma geliştirme ve model geliştirme ortamı olmasıdır. Modelimizi eğitmek için Google Colab ortamını seçmemizin bir diğer nedeni de eğitim sırasındaki donanımsal engelleri kolayca aşabileceğimizi düşünmemizdir.

Çalışmalarımız kapsamında teknofest tarafından sağlanan Sağlık Bakanlığına ait verilerin eğitilmesi için uygun ortam olarak Google Colab, algoritma olarak ta YOLOv4- DarkNet mimarisi kullanılmaya karar verilmiş ve eğitime başlanmıştır.

Eğitim sürecinde Sağlık Bakanlığımızın bize sağladığı Train isimli veri seti ile Google Colaboratory bulut hizmetinde YOLOv4 algoritması eğitilmiş ve nesne algılama yöntemi ile 6 sınıf içeren karın bölgesi görüntüleri kullanılmıştır. Eğitim için zamanımızın az kalması nedeniyle Veri setinde test oranı %16 tutulmuş ve eğitim verisinin çok olması tercih edilmiştir.

2. Özgünlük

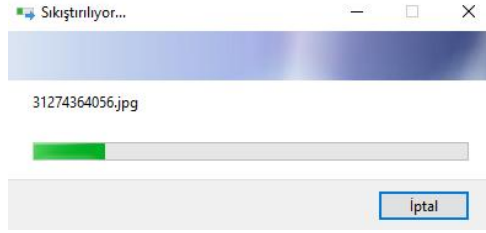
Projemizde yapmaya çalıştığımız “Bilgisayarlı Görüyle Karın Bölgesi (Abdomen) Hastalık Tespiti” modelimizi oluştururken çalışmamızı bölümlere ayırarak aşama aşam ilerlemeyi tercih ettik. Bu sayede daha sistemli çalışabilmenin önünü açmış olabileceğimizi düşündük. İlk olarak bu bölümlerden bahsedecek olursak:

- Verisetini Hazırlama
- Eğitim
- Test
- İyileştirme

Çalışmamızı bu sıra ile gerçekleştirdik. Bu da bizim sadece bu çalışma için geliştirdiğimiz en özgün yön oldu. Halihazırda kullanılmakta olan algoritmaları inceledik. Nesne Algılama Algoritmalarını ve bu konuda kullanacağımız platformları inceledik. Yolo Ver 4.0 platformu bizim çalışmamızı gerçekleştirdiğimiz platform oldu. Tabi bu platformun bazı dezavantajları vardı. Mesela en önemlisi “Tensorboard” isimli görsel sonuçlar ve eğitim sonuçlarını görebildiğimiz görsel grafikler yerine daha basit görselleri kullanmak zorunda kaldık. Bunun temel sebebi proje için zamanımızın az olması idi. Bu kısa zamanda çoğu kavramı öğrenmek için fazla zamanımız yoktu ve tabiri caizse nokta atışı yapmak zorundaydık. Dolayısıyla da Yolo Ver 4.0 ile tensorboard’u kullanamayacağımızı anladığımızda rapor teslim süresine çok yaklaştığımız için yöntem değişikliğine vaktimiz kalmadı. Fakat bu projeyi bu şekilde de olsa başarılı bir şekilde sonuçlandırdığımızı görmek Burobot takımımız için gurur verici bir çalışmaydı.

3. Sonuçlar ve İnceleme

Eğitime başlamadan önce verileri Darknet üzerinde eğitebilmek için drive'a atarak sıkıştırdık.



Resim 1. Dosyaların darknet için sıkıştırılması.

Dosyalarımızı darknet e attıktan sonra burada eğitim için kullanacağımız sistemin özellikleri aşağıdaki resimde görülmektedir.

Sistem Özellikleri:

```
[1] %cat /etc/lsb-release

DISTRIB_ID=Ubuntu
DISTRIB_RELEASE=18.04
DISTRIB_CODENAME=bionic
DISTRIB_DESCRIPTION="Ubuntu 18.04.5 LTS"
```

Resim 2. Darknet sistem özellikleri.

Sistem üzerindeki depolama alanını güncelledik.



Resim 3. Depolama alanını güncelleme.

Daha sonra ise sistem üzerindeki konumumuzu öğrenip darknet klasörümüzü zipten çıkardık. Daha sonra dos2unix indirip dosyaları unix formatına dönüştürdük ve darknet dosyalarımızı çalıştırdık. Darknet testini uygulayıp eğitim sonuçlarına bi bakış yaptık. Ve ilk denemede bir hata aldık yada eğitim gerçekleşmedi. Bu durumu değiştirmek ve eğitimi başarıya ulaştırmak için çalışmaya devam ettik.

```
[8] sudo apt install dos2unix

Reading package lists... Done
Building dependency tree
Reading state information... Done
The following package was automatically installed and is no longer required:
  libnvidia-common-460
Use 'sudo apt autoremove' to remove it.
The following NEW packages will be installed:
  dos2unix
0 upgraded, 1 newly installed, 0 to remove and 61 not upgraded.
Need to get 351 kB of archives:
After this operation, 1,267 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu bionic/universe amd64 dos2unix amd64 7.3.4-3 [351 kB]
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed, so the dialog based frontend cannot be used. at /usr/share/perl5/Debconf/FrontEnd/Dialog.pm line 76, <= line 1.)
debconf: falling back to frontend: Readline
debconf: unable to initialize frontend: Readline
debconf: (This frontend requires a controlling tty.)
debconf: falling back to frontend: Teletype
dpkg-preconfigure: unable to re-open stdin:
Selecting previously unselected package dos2unix.
(Reading database ... 155039 files and directories currently installed.)
Preparing to unpack .../dos2unix_7.3.4-3_amd64.deb ...
Unpacking dos2unix (7.3.4-3) ...
Setting up dos2unix (7.3.4-3) ...
Processing triggers for man-db (2.8.3-2ubuntu1) ...

[9] unzip "/content/drive/mydrive/custom_model/darknet.zip"
Inflating: darknet/src/naopool_layer.c
Inflating: darknet/src/naopool_layer.h
Inflating: darknet/src/naopool_layer_kernels.cu
Inflating: darknet/src/network.c
Inflating: darknet/src/network.h
Inflating: darknet/src/network_kernels.cu
Inflating: darknet/src/nightmare.c
Inflating: darknet/src/normalization_layer.c
Inflating: darknet/src/normalization_layer.h
Inflating: darknet/src/option_list.c
Inflating: darknet/src/option_list.h
Inflating: darknet/src/parser.c
Inflating: darknet/src/parser.h
Inflating: darknet/src/region_layer.c
Inflating: darknet/src/region_layer.h
Inflating: darknet/src/region_layer_kernels.cu
Inflating: darknet/src/region_layer.h
Inflating: darknet/src/region_old_layer.c
Inflating: darknet/src/region_old_layer.h
Inflating: darknet/src/representation_layer.c
Inflating: darknet/src/representation_layer.h
Inflating: darknet/src/rnn.c
Inflating: darknet/src/rnn_layer.c
Inflating: darknet/src/rnn_layer.h
Inflating: darknet/src/rnn_vld.c
```

Resim 4. (a) Darknet klasörünü zipten çıkarma (b) dos2unix indirme

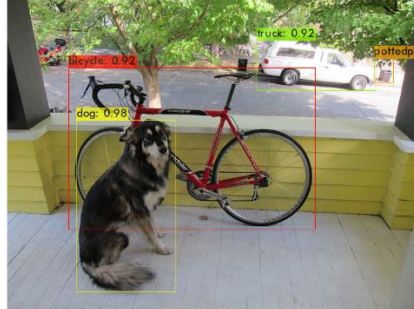
```
ifind -type f -print0 | xargs -0 dos2unix

dos2unix: converting file ./custom_data/labels/383080383462.txt to unix format...
dos2unix: converting file ./custom_data/labels/383080383463.txt to unix format...
dos2unix: converting file ./custom_data/labels/383080383464.txt to unix format...
dos2unix: converting file ./custom_data/labels/383080383465.txt to unix format...
dos2unix: converting file ./custom_data/labels/383080383466.txt to unix format...
dos2unix: converting file ./custom_data/labels/383080383467.txt to unix format...
dos2unix: converting file ./custom_data/labels/383080383468.txt to unix format...
dos2unix: converting file ./custom_data/labels/383080383469.txt to unix format...
dos2unix: converting file ./custom_data/labels/383080383470.txt to unix format...
dos2unix: converting file ./custom_data/labels/383080383471.txt to unix format...
dos2unix: converting file ./custom_data/labels/383080383472.txt to unix format...
dos2unix: converting file ./custom_data/labels/383080383473.txt to unix format...
dos2unix: converting file ./custom_data/labels/383080383474.txt to unix format...
dos2unix: converting file ./custom_data/labels/383080383475.txt to unix format...
dos2unix: converting file ./custom_data/labels/383080383476.txt to unix format...
dos2unix: converting file ./custom_data/labels/383080383477.txt to unix format...
dos2unix: converting file ./custom_data/labels/383080383478.txt to unix format...
dos2unix: converting file ./custom_data/labels/383080383479.txt to unix format...
dos2unix: converting file ./custom_data/labels/383080383480.txt to unix format...
```

Resim 5 . Dosyaları unix e çevirme

```
lnake
./src/ln2col_kernels.cu(1364): warning: unrecognized #pragma in device code
./src/ln2col_kernels.cu(1361): warning: unrecognized #pragma in device code
./src/ln2col_kernels.cu(1364): warning: unrecognized #pragma in device code
./src/ln2col_kernels.cu(1369): warning: unrecognized #pragma in device code
nvcc -gencode arch=compute_35,code=sm_35 -gencode arch=compute_50,code=sm_50,compute_50 -gencode arch=compute_52,code=sm_52,compute_52 -gencode arch=compute_61,code=sm_61,compute_61 -I
nvcc warning: The 'compute_35', 'compute_37', 'compute_50', 'sm_35', 'sm_37' and 'sm_50' architectures are deprecated, and may be removed in a future release (Use -xno-deprecated-gpu-target
nvcc warning: The 'compute_35', 'compute_37', 'compute_50', 'sm_35', 'sm_37' and 'sm_50' architectures are deprecated, and may be removed in a future release (Use -xno-deprecated-gpu-target
./src/bias_kernels.cu(1886): warning: variable "out_index" was declared but never referenced
./src/bias_kernels.cu(1130): warning: variable "step" was set but never used
./src/bias_kernels.cu(1736): warning: variable "stage_id" was declared but never referenced
./src/bias_kernels.cu(1886): warning: variable "out_index" was declared but never referenced
./src/bias_kernels.cu(1130): warning: variable "step" was set but never used
./src/bias_kernels.cu(1736): warning: variable "stage_id" was declared but never referenced
./src/bias_kernels.cu(1886): warning: variable "out_index" was declared but never referenced
```

Resim 6. Darknet dosyalarını çalıştırma



Resim 7. Darknet testi

```
[50] !rm /content/darknet/backup -r
```

```
[51] !ln -s /content/drive/"My Drive"/burobot_weights/backup /content/darknet
```

Resim 8. Drive backup bağlantısı

```

./darknet detector train custom_data/data.data burobot_yolov4.cfg yolov4.conv.137 -map -dont_show

v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 139 Avg (IOU: 0.000000), count: 1, class_loss = 4295.945801, iou_loss = 0.000000, tot
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 150 Avg (IOU: 0.000000), count: 1, class_loss = 993.176208, iou_loss = 0.000000, tot
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 161 Avg (IOU: 0.000000), count: 1, class_loss = 225.700516, iou_loss = 0.000000, tot
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 139 Avg (IOU: 0.000000), count: 1, class_loss = 4296.375977, iou_loss = 0.000000, tc
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 150 Avg (IOU: 0.000000), count: 1, class_loss = 1007.125732, iou_loss = 0.000000, tc
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 161 Avg (IOU: 0.000000), count: 1, class_loss = 227.309708, iou_loss = 0.000000, tot
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 139 Avg (IOU: 0.000000), count: 1, class_loss = 4287.925781, iou_loss = 0.000000, tc
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 150 Avg (IOU: 0.000000), count: 1, class_loss = 999.976685, iou_loss = 0.000000, tot
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 161 Avg (IOU: 0.000000), count: 1, class_loss = 227.016968, iou_loss = 0.000000, tot
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 139 Avg (IOU: 0.000000), count: 1, class_loss = 4284.437988, iou_loss = 0.000000, tc
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 150 Avg (IOU: 0.000000), count: 1, class_loss = 992.543335, iou_loss = 0.000000, tot
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 161 Avg (IOU: 0.000000), count: 1, class_loss = 226.315414, iou_loss = 0.000000, tot
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 139 Avg (IOU: 0.000000), count: 1, class_loss = 4303.863281, iou_loss = 0.000000, tc
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 150 Avg (IOU: 0.000000), count: 1, class_loss = 996.173584, iou_loss = 0.000000, tot
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 161 Avg (IOU: 0.000000), count: 1, class_loss = 224.992157, iou_loss = 0.000000, tot
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 139 Avg (IOU: 0.000000), count: 1, class_loss = 4288.161621, iou_loss = 0.000000, tc
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 150 Avg (IOU: 0.000000), count: 1, class_loss = 994.339417, iou_loss = 0.000000, tot
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 161 Avg (IOU: 0.000000), count: 1, class_loss = 225.818344, iou_loss = 0.000000, tot
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 139 Avg (IOU: 0.000000), count: 1, class_loss = 4279.718262, iou_loss = 0.000000, tc
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 150 Avg (IOU: 0.000000), count: 1, class_loss = 998.217224, iou_loss = 0.000000, tot
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 161 Avg (IOU: 0.000000), count: 1, class_loss = 227.146225, iou_loss = 0.000000, tot
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 139 Avg (IOU: 0.000000), count: 1, class_loss = 4267.389160, iou_loss = 0.000000, tc
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 150 Avg (IOU: 0.000000), count: 1, class_loss = 989.875671, iou_loss = 0.000000, tot
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 161 Avg (IOU: 0.000000), count: 1, class_loss = 225.678375, iou_loss = 0.000000, tot
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 139 Avg (IOU: 0.000000), count: 1, class_loss = 4272.511230, iou_loss = 0.000000, tc
v3 (iou loss, Normalizer: (iou: 0.07, obj: 1.00, cls: 1.00) Region 150 Avg (IOU: 0.000000), count: 1, class_loss = 991.314148, iou_loss = 0.000000, tot

```

Resim 9. Eğitim süreci.

Yolo v4 modelinin bizden skale edilmiş (indirgenmiş) veri istemesinden dolayı eğitim sonuçlarımızın hatalı çıktığını anladık. Verileri aşağıdaki kodlar ile 0 ile 1 arasına indirgedik.

```

Verilen 0 ile 1 arasına indirgenmesi.py x
1 import os
2 from sklearn.preprocessing import MinMaxScaler
3 import pandas as pd
4 import time
5 scl = MinMaxScaler()
6 cl = []
7 k1 = []
8 k2 = []
9 k3 = []
10 k4 = []
11
12 os.chdir("E:\\darknet\\custom_data\\labels\\")
13
14 for f1 in os.listdir():
15     print(f1)
16     f = open(os.getcwd() + "\\\" + f1, "r")
17     r = f.read()
18     r = r.split("\n")
19     c1.append(r[0])
20     k1.append(r[1])
21     k2.append(r[2])
22     k3.append(r[3])
23     k4.append(r[4])
24     f.close()
25
26 print("=====")
27 time.sleep(2)
28 df = pd.DataFrame({"k1":k1,"k2":k2,"k3":k3,"k4":k4})
29 df = scl.fit_transform(df)
30
31 for i, f2 in enumerate(os.listdir()):
32     os.remove(os.getcwd() + "\\\" + f2)
33     f3 = open(f2, "w")
34     f3.write(str(c1[i][0]) + " " + str(df[i][0]) + " " + str(df[i][1]) + " " + str(df[i][2]) + " " + str(df[i][3]) + " " + str(df[i][4]) + "\n")
35     f3.close()
36     print("OLUSTURULDU: " + f2)
37

```

Resim 10. Verilerin 0 ile 1 arasına indirgenmesi.

Bazı sınıfların index lerinin bu hataya sebep olduğunu anladık ve bu durumu düzeltmek için de aşağıdaki kodlarla bu sorunun üstesinden geldik.

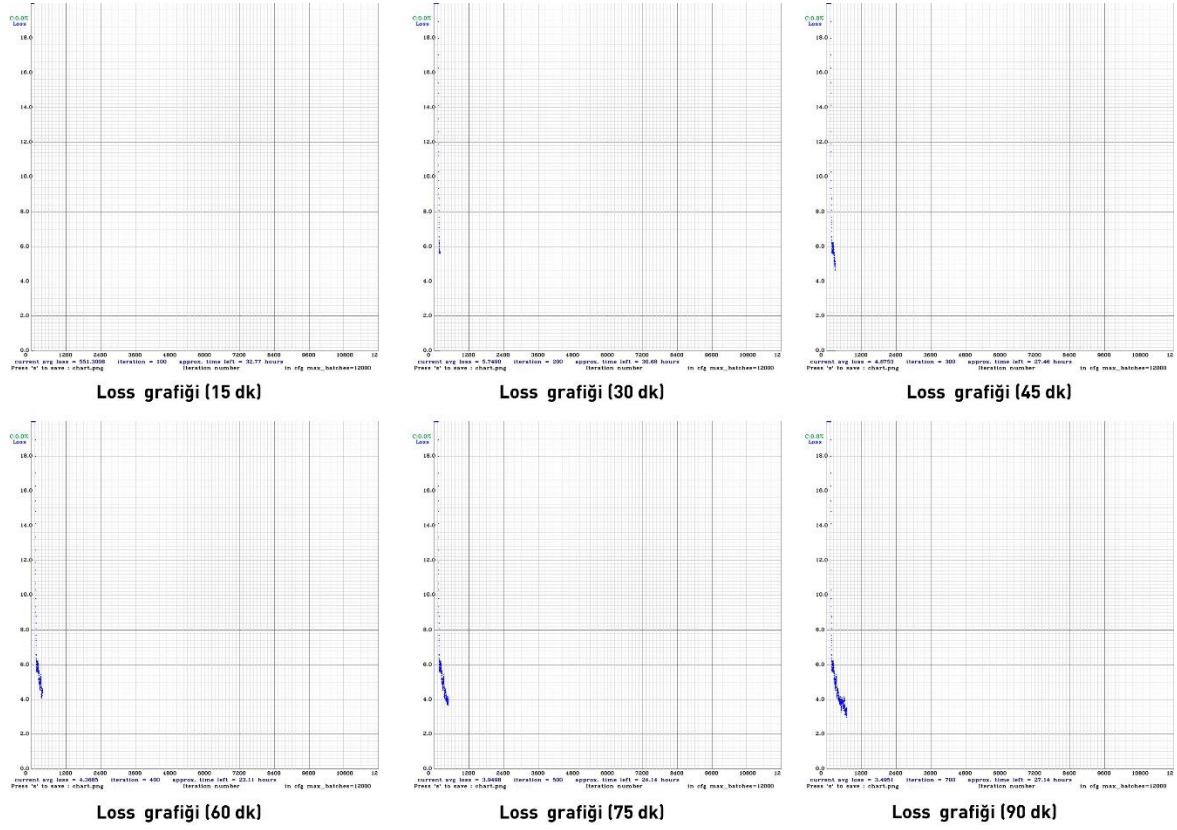
```

E:\Yran\Yaslan Kodları\Bozuk classların düzeltilmesi.py
1 import os
2
3 os.chdir("E:\\darknet\\custom_data\\labels\\")
4
5
6 for f1 in os.listdir():
7     f = open(os.getcwd() + "\\\" + f1, "r")
8     r = f.read()
9     c = int(r[0])
10    c = c-1
11    r = str(c) + r[1:]
12    f.close()
13    os.remove(os.getcwd() + "\\\" + f1)
14    f3 = open(os.getcwd() + "\\\" + f1, "w")
15    f3.write(r)
16    f3.close()
17    print("Olusturuldu:" + f1)

```

Resim 11. Class index lerini düzeltme.

Eğitimi tekrar denediğimizde aşağıdaki sonuçlara ulaştık. İlk resimde loss (kayıp) grafiği görülmektedir.



Resim 12. Kayıpların(Loss) grafikleri .

Modelimiz map sonuçlarına göre ise bir sınıfta %30 başarı göstermiş görünüyor. Diğer sınıflarda eğitime devam etmekteyiz. Düşük sonuç çıkmasının ana sebebi eğitim modelimizi nasıl geliştireceğimizi yeni öğrenmiş olmamız ve rapor için zaman kalmamasıdır. Raporun son günü itibarıyla sadece 90 dk eğitim yapabildik Uzun süreli bir eğitim ile (yaklaşık 24 saat) daha yüksek sonuçlara ulaşacağımızı düşünmekteyiz. Çünkü eğitim sırasında meydana gelen tüm hatalar düzeltilmiş ve hata uyarısı alınmamıştır. Eğitim sonucunun düşük çıkmasının sebebi zamanımızın yetmemesidir. Final yarışması için kalan 4 haftalık zamanda da modelin eğitime ve teste devam edeceğiz. Final aşamasına kadar eğitimlerdeki hataları giderip modelimizi tam anlamıyla eğitmeyi ve testlerdeki başarı oranımızı yükseltmeyi düşünüyoruz. Bu konuda kaydettiğimiz gelişme, bu öğrenme modelini geliştireceğimizin en güzel kanıtı olmaktadır.

```

./darknet detector map /content/darknet/custom_data/data.burobot_yolov4.cfg /content/drive/MyDrive/burobot_weights/backup/burobot_yolov4_last.wei
CUDA-version: 11010 (11020), cuDNN: 7.6.5, GPU count: 1
OpenCV version: 3.2.0
0 : compute_capability = 750, cudnn_half = 0, GPU: Tesla T4
net.optimized_memory = 0
mini_batch = 1, batch = 64, time_steps = 1, train = 0
layer filters size/strd(dil) input output
0 Create CUDA-stream - 0
Create cudnn-handle 0
conv 32 3 x 3/ 1 416 x 416 x 3 -> 416 x 416 x 32 0.299 BF
1 conv 64 3 x 3/ 2 416 x 416 x 32 -> 208 x 208 x 64 1.595 BF
2 conv 64 1 x 1/ 1 208 x 208 x 64 -> 208 x 208 x 64 0.354 BF
3 route 1 -> 208 x 208 x 64
4 conv 64 1 x 1/ 1 208 x 208 x 64 -> 208 x 208 x 64 0.354 BF
5 conv 32 1 x 1/ 1 208 x 208 x 64 -> 208 x 208 x 32 0.177 BF
6 conv 64 3 x 3/ 1 208 x 208 x 32 -> 208 x 208 x 64 1.595 BF
7 Shortcut Layer: 4, wt = 0, wn = 0, outputs: 208 x 208 x 64 0.003 BF
8 conv 64 1 x 1/ 1 208 x 208 x 64 -> 208 x 208 x 64 0.354 BF
9 route 0 2 -> 208 x 208 x 128
10 conv 64 1 x 1/ 1 208 x 208 x 128 -> 208 x 208 x 64 0.709 BF
11 conv 128 3 x 3/ 2 208 x 208 x 64 -> 104 x 104 x 128 1.595 BF
12 conv 64 1 x 1/ 1 104 x 104 x 128 -> 104 x 104 x 64 0.177 BF
13 route 11 -> 104 x 104 x 128
14 conv 64 1 x 1/ 1 104 x 104 x 128 -> 104 x 104 x 64 0.177 BF
15 conv 64 1 x 1/ 1 104 x 104 x 64 -> 104 x 104 x 64 0.089 BF
16 conv 64 3 x 3/ 1 104 x 104 x 64 -> 104 x 104 x 64 0.797 BF
17 Shortcut Layer: 14, wt = 0, wn = 0, outputs: 104 x 104 x 64 0.001 BF
18 conv 64 1 x 1/ 1 104 x 104 x 64 -> 104 x 104 x 64 0.089 BF
19 conv 64 3 x 3/ 1 104 x 104 x 64 -> 104 x 104 x 64 0.797 BF
20 Shortcut Layer: 17, wt = 0, wn = 0, outputs: 104 x 104 x 64 0.001 BF
21 conv 64 1 x 1/ 1 104 x 104 x 64 -> 104 x 104 x 64 0.089 BF
22 route 21 12 -> 104 x 104 x 128
1 dk. 28 sn. tamamlanma zamanı: 15:10

```

Resim 13. Map sonuçları.

```

!./darknet detector test /content/darknet/custom_data/data.data burobot_yolov4.cfg /content/drive/MyDrive/burobot_weights/backup/burobot_yolov4_last.weights
import cv2
import matplotlib.pyplot as plt
%matplotlib inline

img = cv2.imread('predictions.jpg')
h, w = img.shape[:2]
resized_img = cv2.resize(img, (3*w, 3*h), interpolation = cv2.INTER_CUBIC)

fig = plt.gcf()
fig.set_size_inches(18, 10)
plt.axis("off")
plt.imshow(cv2.cvtColor(resized_img, cv2.COLOR_BGR2RGB))
plt.show()

```

Resim 14. Test sayfası.

```

a burobot_yolov4.cfg /content/drive/MyDrive/burobot_weights/backup/burobot_yolov4_last.weights /content/darknet/custom_data/images/10489533780.jpg

INTER_CUBIC)

```

Resim 15. Test sayfası.

4. Deney ve Eğitim Aşamalarında Kullanılan Veri Setleri

4.1. Verisetinin Eğitime Hazırlanması

Veriseti olarak sadece Sağlık Bakanlığı'nın bize sağlamış olduğu verileri kullandık. Bu veriseti haricinde başka bir veriseti kullanmadık. Verisetimiz üzerinde ilk yaptığımız işlem verisetindeki bozuk dosyaların temizlenmesi oldu.

Bozuk dosyaları dataframe üzerinden temizledik ve bu dosyaları sildik.

```

Bozuk Dosyaların DataFrame üzerinden temizlenmesi.py x
1 import pandas as pd
2 import os
3 df = pd.read_csv("E:/Train/Training/DATA3.csv")
4 os.chdir("E:/Train/Training")
5 df.columns = ["olgu", "kesit", "tip", "sinif", "koordinatlar"]
6 olgu_l = []
7 print(df.tail())
8 for f in os.listdir()[:-8]:
9     f = f+"/"+os.listdir(os.getcwd() + "\\") + f)[0]
10    olgu_l.append(f)
11
12 olgu = list(df.olgu)
13 uyumsuzlar = list(set(olgu_l).symmetric_difference(set(olgu)))
14 print("=====")
15 for uyumsuz in uyumsuzlar:
16
17     df = df[df.olgu != uyumsuz]
18
19 print(df.tail())
20 os.chdir("E:/Train/Training")
21 df.to_csv('DATA4.csv', index=False)
22
23

```

Resim 16. Bozuk dosyaların dataframe üzerinden kaldırılması

```
Bozuk Dosyaların Silimi.py X

import os
import shutil

os.chdir("E:\\Train\\Training\\")
main_files = os.listdir()[1:182]
curd = ""
error_f = []
old_d = ""
errs = []
errs2 = []
for i,f in enumerate(main_files):
    os.chdir("E:\\Train\\Training\\")
    curd = os.getcwd() + "\\\" + f
    if curd[-3:] != ".py" and curd[-5:] != ".xlsx" and curd[-4:] != ".csv" and curd[-4:] != ".txt":
        os.chdir(curd)
    else:
        continue
    for f2 in os.listdir():
        try:
            curd = "E:\\Train\\Training\\" + f
            curd = curd + "\\\" + f2
            os.chdir(curd)
        except:
            errs2.append(curd)
        for f3 in os.listdir():
            if f3[-4:] == ".dcm":
                try:
                    del_f = ""
                    print("=====")
                    print("BOZUK DOSYA TESPİT EDİLDİ:")
                    del_f = os.getcwd().split("E:\\Train\\Training\\")[1]
                    del_f = del_f.replace("\\", "/")
                    os.chdir("C:\\")
                    shutil.rmtree("E:\\Train\\Training\\" + del_f.split("/") [0])
                    print(del_f)
                    print("BOZUK DOSYA KALDIRILDI:" + del_f)
                    print("=====")
                    break
                except:
                    errs.append("E:\\Train\\Training\\" + del_f.split("/") [0])
os.chdir("E:\\Train\\Training\\")
```

Resim 17. Bozuk dosyaların silinmesi

Veriseti üzerinde ikinci olarak yaptığımız iş ise dataframe üzerinde olmayan dosyaların silinmesi oldu.

```
E:\Train\Veri\Veri Kodları\DataFrame üzerinde olmayan dosya olarak dosyaların silimi.py

1 import os
2 import pandas as pd
3
4
5
6 os.chdir("E:\\Train\\Training\\")
7 df = pd.read_csv("E:\\Train\\Training\\DATA5.csv")
8 df2 = df
9
10 for i,f in enumerate(os.listdir()[1:182]):
11     os.chdir("E:\\Train\\Training\\")
12     curd = os.getcwd() + "\\\" + f
13     for f2 in os.listdir():
14         curd = "E:\\Train\\Training\\" + f
15         curd = curd + "\\\" + f2
16         os.chdir(curd)
17         for f3 in os.listdir():
18             if not os.path.exists(os.getcwd() + "\\\" + f3[-4:] + ".txt"):
19                 os.remove(os.getcwd() + "\\\" + f3)
20                 df = df[df.kesit != df2["kesit"][i]]
21                 print("DOSYA NEW DATAFRAME NEW DOSYALARDAN KALDIRILDI:" + os.getcwd() + "\\\" + f3)
22
23 os.chdir("E:\\Train\\Training\\")
24 df.to_csv("DATA6.csv", index=False)
25
26
```

Resim 18. Dataframe üzerinde görünmeyen dosyaların temizlenmesi.

Veriseti ile eğitime geçmeden önce tüm karmaşık yapıları düzenlemek ve basitleştirmek adına işimize yaramayacak olan sınıfları temizleyerek bize zaman kaybettirecek olan karmaşık yapıyı basitleştirdik.

```
E:\Train\Veri\Veri Kodları\Gereksiz sınıfların silinmesi.py

1 import os
2 import pandas as pd
3
4
5 df = pd.read_csv("E:\\Train\\Training\\DATA4.csv")
6
7
8 os.chdir("E:\\Train\\Training\\")
9 siniflar = {"Akut apandisit ile uyumlu": "1",
10            "Akut kolelisit ile uyumlu": "2",
11            "Akut pankreatit ile uyumlu": "3",
12            "Babreke testi": "4",
13            "Üreter testi": "4",
14            "Akut divertikülit ile uyumlu": "5",
15            "Abdominal aort anevrizması": "6",
16            "Abdominal Aorta": "5",
17            "Abdominal aort diseksiyonu": "6"}
18
19 df2 = df
20 for i, sinif in enumerate(df["sinif"]):
21     if sinif not in siniflar.keys():
22         try:
23             olgu = str(df["olgu"][i])
24             os.remove("E:\\Train\\Training\\" + olgu.replace("/", "\\") + "\\\" + df["kesit"][i] + ".jpg")
25             df = df[df.kesit != df2["kesit"][i]]
26             print("Dosya DataFrame den ve resimlerden silindi")
27         except:
28             print("ATLANDI:", "E:\\Train\\Training\\" + olgu.replace("/", "\\") + "\\\" + df2["kesit"][i] + ".jpg")
29             df = df[df.kesit != df2["kesit"][i]]
30
31 os.chdir("E:\\Train\\Training\\")
32 df.to_csv("DATA5.csv", index=False)
```

Resim 19. Gereksiz sınıfların silinmesi.

Resimler için gereksiz sınıflar temizlendikten sonra eğitim öncesinde tüm resimler dicom(.dcm) formatlı olduğundan bunları .jpg formatına dönüştürmeye karar verdik. Çünkü işlemlerimize .dcm ile devam etmemiz mümkün değildi. Bu dönüştürmeyi “pydicom” isimli kütüphane ile gerçekleştirdik . Dönüştürme öncesi bu kütüphaneyi indirdik ve dönüştürme işlemini aşağıdaki kodlarla gerçekleştirdik.

```
In [5]: pip install pydicom
```

Resim 20. pydicom kütüphanesinin yüklenmesi.

```
import os
import pydicom
import numpy as np
from PIL import Image
import time

os.chdir("E:\\Train\\Training\\")
main_files = os.listdir()[1:182]
curd = ""
error_f = []
for i,f in enumerate(main_files):
    os.chdir("E:\\Train\\Training\\")
    curd = os.getcwd() + "\\\" + f
    os.chdir(curd)
    for f2 in os.listdir():
        curd = "E:\\Train\\Training\\" + f
        curd = curd + "\\\" + f2
        os.chdir(curd)
        for f3 in os.listdir():
            try:
                if f3[-4:] == ".dcm":
                    print("=====")
                    print("DICOM TESPİT EDİLDİ:" + os.getcwd() + "\\\" + f3)
                    img = pydicom.dcmread(os.getcwd() + "\\\" + f3,force=True)

                    img = img.pixel_array.astype(float)

                    simg = (np.maximum(img, 0)/img.max())*255

                    fimg= np.uint8(simg)

                    fimg = Image.fromarray(fimg)

                    fimg.save(os.getcwd() + "\\\" + f3[:-4] + ".jpg")
                    os.remove(os.getcwd() + "\\\" + f3)
                    print("DICOM CEVİRİLDİ:"+f3)
                    print("=====")
            except:
                error_f.append(os.getcwd() + "\\\" + f3)
print("=====")
print("Bozuk dosyalar:")
for error in error_f:
    os.chdir("E:\\Train")
    fi = open("Yapay_zeka_bozuk_DCOM.txt", "w")
    fi.write(error)
    fi.write("\n")
time.sleep(10)
os.system('shutdown -s')
```

Resim 21. Tüm resimlerin “dcm to jpg” dönüşümünü yapan kodlar.

Veriseti üzerinde son yaptığımız işlem verisetini Train(Eğitim) ve Test olarak ayırmak oldu. Burada verisetini ayırmak için çeşitli oranlar, bilgiler internet üzerinde çeşitli yazılar bulduk. Fakat biz kendi projemizde test oranımızı %16 olarak belirledik.

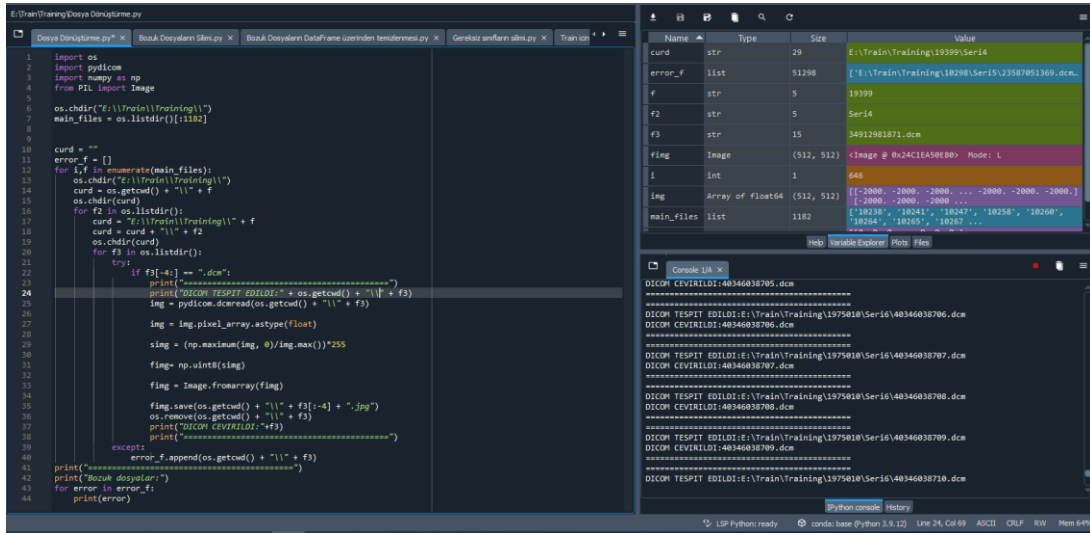
The screenshot shows a Python IDE with a script for splitting data into train and test sets. The script uses a random seed and a 16% test ratio. The console window displays the execution results, showing the number of files in the train and test sets.

Name	Type	Size	Value
sinif	str	5	Kolon
siniflar	dict	9	{'Akut miyokard infarktüsü ile uyumlu':1, 'Akut koroner arter hastalığı ile uyumlu':2, 'A ...
test	str	47	E:\\Train\\Training\\1188321\\Ser12\\3850000071.jpg
test_item_paths	list	3057	[E:\\Train\\Training\\10238\\Ser14\\3983803...
test_size	int	1	3057
test_yuzdesi	int	1	16
train	str	45	E:\\Train\\Training\\126488\\Ser12\\26373981336.jpg
train_item_paths	list	10058	[E:\\Train\\Training\\1188321\\Ser12\\3850...
train_size	int	1	10058

The console output shows the following messages:

```
7988 DOSYALAR TRAIN VE TEST OLARAK AYRIYILYOR
7989 DOSYALAR TRAIN VE TEST OLARAK AYRIYILYOR
7990 DOSYALAR TRAIN VE TEST OLARAK AYRIYILYOR
7991 DOSYALAR TRAIN VE TEST OLARAK AYRIYILYOR
7992 DOSYALAR TRAIN VE TEST OLARAK AYRIYILYOR
7993 DOSYALAR TRAIN VE TEST OLARAK AYRIYILYOR
7994 DOSYALAR TRAIN VE TEST OLARAK AYRIYILYOR
7995 DOSYALAR TRAIN VE TEST OLARAK AYRIYILYOR
7996 DOSYALAR TRAIN VE TEST OLARAK AYRIYILYOR
7997 DOSYALAR TRAIN VE TEST OLARAK AYRIYILYOR
7998 DOSYALAR TRAIN VE TEST OLARAK AYRIYILYOR
7999 DOSYALAR TRAIN VE TEST OLARAK AYRIYILYOR
```

Resim 22. Verilerin train ve test olarak ayrılması.



Resim 23. Dosya dönüştürme işlemleri.

5. Referanslar

- [1] Murat, S., 2021, *İnsansız hava aracı görüntülerinden derin öğrenme yöntemleriyle nesne tanıma* (Master's thesis, Maltepe Üniversitesi, Lisansüstü Eğitim Enstitüsü).
- [2] C.-Y. W. H.-Y. M. L. Alexey Bochkovskiy, 2020, *YOLOv4: Optimal Speed and Accuracy of Object Detection*.
- [3] P. F. T. B. Olaf Ronneberger, 2015, *U-Net: Convolutional Networks for Biomedical Image Segmentation*, p. 3.
- [4] <https://globalaihub.com/blog/google-colab-nedir-ve-nasil-kullanilir> adresine 30.06.2022 tarihinde erişilmiştir.
- [5] Atik et al., (2022). *Comparison of YOLO Versions for Object Detection from Aerial Images*, *International Journal of Environment and Geoinformatics (IJEgeo)*, 9(2):087-093 doi. 10.30897/ijegeo.1010741
- [6] <https://www.udemy.com/course/bilgisayarl-goru-yolov4-ile-nesne-tanma/> adresine Nisan-Haziran 2022 tarihleri arasında erişilmiştir.
- [7] Doç.Dr.Emrah HANÇER (MAKÜ Bucak Teknoloji Fakültesi Yazılım Mühendisliği Bölüm Başkanı) , Kişisel Görüşme, 17 Haziran 2022.
- [8] Ali Tezcan SARIZEYBEK (MAKÜ Bucak Teknoloji Fakültesi Yazılım Mühendisliği Araştırma Görevlisi) , Kişisel Görüşme, 17 Haziran 2022.
- [9] Ömer Can ESKİCİOĞLU (MAKÜ Bucak Teknoloji Fakültesi Yazılım Mühendisliği Araştırma Görevlisi) , Kişisel Görüşme, 17 Haziran 2022.
- [10] <https://www.udemy.com/course/python-ile-makine-ogrenimi/> adresine Nisan- Haziran 2022 tarihleri arasında erişilmiştir.
- [11] <https://www.udemy.com/course/makine-ogrenmesi/> adresine Nisan- Haziran 2022 tarihleri arasında erişilmiştir.
- [12] <https://www.udemy.com/course/veri-bilimine-giris/> adresine Nisan- Haziran 2022 tarihleri arasında erişilmiştir.
- [13]<https://www.udemy.com/course/derin-ogrenme-ile-goruntu-isleme-python-opencv-ve-keras/> adresine Nisan- Haziran 2022 tarihleri arasında erişilmiştir.
- [14]<https://www.youtube.com/channel/UC0GwTXvSheFR7HVxHVwiZUG> adresine Nisan-Haziran 2022 tarihleri arasında erişilmiştir.
- [15]<https://www.youtube.com/channel/UCmwYAzIl3NvwG2xKNcZDjTA> adresine Nisan-Haziran 2022 tarihleri arasında erişilmiştir.
- [16]https://www.youtube.com/channel/UCAnbdATUWOWo-we_dbWNZw adresine Nisan-Haziran 2022 tarihleri arasında erişilmiştir.